Reviewer Pack — Minimal Local Coherence Rank Correlation with Resolution Pro...

Entry f20e7e15ea6a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:31:03 UTC

Minimal Local Coherence Rank Correlation with Resolution Proof Width via Sheaf Theory

Entry ID: `f20e7e15ea6a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 03:31:03 UTC

1. Conjecture

Field A (mathematical branch): Sheaf Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Boolean circuit φ with depth d and n inputs, the minimal local coherence rank of its associated sheaf is linearly correlated with its resolution proof width w(φ), such that the minimal local coherence rank r(φ) = Θ(w(φ)).

Rationale (proposer's reasoning):

> Sheaf theory provides a framework to study the structure of spaces locally, which may reveal hidden complexities in circuits. The minimal local coherence rank measures the complexity of global sections of sheaves, and its correlation with resolution proof width could potentially expose new insights into the hardness of satisfiability problems.

Taxonomy category: `Sheaf_Theory × Resolution_Proof_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1618bcab5a51940b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Boolean circuit φ with depth d and n inputs, if |r(φ) - w(φ)| is within Θ(f(n)) for all circuits φ where r(φ) is the minimal local coherence rank and w(φ) is the resolution proof width, then accept the conjecture.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal local coherence rank AND Sheaf Theory AND resolution proof complexity - 0-linear correlation AND sheaf theory AND resolution proof width - resolution proof width AND sheaf theory AND Boolean circuit depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, d):
        if n == 1:
            return ['0'] * (2 ** d - 1)
        else:
            circuits = [generate_circuit(n-1, d//2)]
            for i in range(1 << (d//2)):
                circuits.append([f'({circuits[0][i]} {random.choice(["&", "|"])} x{i+1})'])
            return sum(circuits, [])

    def compute_sheaf_rank(circuit):
        # Placeholder function to simulate the computation
        return len(circuit)

    def compute_resolution_width(circuit):
        # Placeholder function to simulate the computation
        return len(circuit) ** 2

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        total_rank = 0
        total_width = 0

        while instances_tested < 30:
            circuit = generate_circuit(n, random.randint(5, 10))
```

```
            rank = compute_sheaf_rank(circuit)
            width = compute_resolution_width(circuit)

            if rank > 0 and width > 0:
                total_rank += rank
                total_width += width
                instances_tested += 1

        if instances_tested == 0:
            return {
                "metric_name": "correlation",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": n_values[-1],
                "conjecture_holds": False,
                "counterexample": "No valid circuits generated"
            }

        mean_rank = total_rank / instances_tested
        mean_width = total_width / instances_tested

        results.append({
            "n": n,
            "mean_rank": mean_rank,
            "mean_width": mean_width
        })

    correlation_coefficient = 0.0
    for result in results:
        correlation_coefficient += (result["mean_rank"] - result["mean_width"]) ** 2

    correlation_coefficient /= len(results)
    correlation_coefficient = math.sqrt(correlation_coefficient)

    return {
        "metric_name": "correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": sum(result["instances_tested"] for result in results),
        "n_max": n_values[-1],
        "conjecture_holds": correlation_coefficient <= 0.1 * n_values[-1],  # Placeholder threshol
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if
```

```
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] or result["instances_tested"] == 0 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] and result["instances_tested"] > 0 for result in resul
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient\" first_failing_seed={f
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 47, in run_trial
    circuit = generate_circuit(n, random.randint(5, 10))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 25, in generate_circ
    circuits = [generate_circuit(n-1, d//2)]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 25, in generate_circ
    circuits = [generate_circuit(n-1, d//2)]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 25, in generate_circ
    circuits = [generate_circuit(n-1, d//2)]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3875709f.py", line 27, in generate_circ
    circuits.append([f'({circuits[0][i]} {random.choice(["&", "|"])} x{i+1})'])
                         ~~~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing. If the support condition is met after re-running the test, then the conjecture can be considered SUPPORTED.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15664
2	propose	ollama_remote	glm4:latest	0	0	14190
3	preregistration	ollama_remote	glm4:latest	0	0	19670
4	novelty	ollama_remote	glm4:latest	0	0	13072
5	novelty	ollama_remote	glm4:latest	0	0	12386
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15741
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	80529
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	70888
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19071
10	judge	ollama_remote	glm4:latest	0	0	12338

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 273549 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f20e7e15ea6a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f20e7e15ea6a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f20e7e15ea6a.tar.gz` (if generated)